

CE802 Worked Solutions (2024 Paper)

Step-by-step explanations, formulas, and calculations for every question.

Prepared from the uploaded 2024 CE802 exam PDF. The solutions below avoid a bare answer-key style and instead show the reasoning used to reach each result.

How to read this document

For each question, the method is explained first, then the intermediate calculations are shown, and only then is the final conclusion stated. Where a question was conceptual rather than numerical, the explanation focuses on the principle being tested and how it applies to the specific exam prompt.

Question 1 - Support vectors from the margin plot

In a support vector classifier, a training point is a support vector if it lies exactly on a margin, or inside the margin. In the diagram, the three drawn contours represent $f(\mathbf{x})=+1$, $f(\mathbf{x})=0$ (decision boundary), and $f(\mathbf{x})=-1$. So we inspect whether each point touches a margin curve or falls in the strip between the two margins.

Point	Reasoning from the figure	Conclusion
A	Circle A lies on the negative margin contour labelled -1.	Support vector
B	Circle B lies on the same negative margin contour.	Support vector
C	Circle C is well inside the negative region, away from the margin.	Not a support vector
D	Triangle D lies on the positive margin contour.	Support vector
E	Triangle E is on the positive side but inside the margin strip, so it still constrains the classifier.	Support vector
F	Triangle F is well outside the positive margin.	Not a support vector
G	Circle G is deep inside the negative region.	Not a support vector
H	Triangle H lies on the positive margin at the far right of the plot.	Support vector
I	Circle I lies on the negative margin contour.	Support vector
J	Triangle J lies on the positive margin contour.	Support vector

Question 1 conclusion

The support vectors are **A, B, D, E, H, I, and J**. The remaining points **C, F, and G** are too far from the margin to determine the classifier.

Question 2 - Linear separability and nonlinear feature maps

A data set is linearly separable if one straight line can place all examples of one class on one side and all examples of the other class on the other side. For the original six points, plotting them shows that no single line can achieve zero classification error. So the original data are **not linearly separable**.

1) Transformation $\phi_1([x_1, x_2]) = [x_1 + 2|x_2|, x_2]$

Original point	Mapped point under ϕ_1
(5, -3)	$(5 + 2 \cdot 3, -3) = (11, -3)$

Original point	Mapped point under phi_1
(-6, -3)	$(-6 + 2 \cdot 3, -3) = (0, -3)$
(2, 7)	$(2 + 2 \cdot 7, 7) = (16, 7)$
(0, 1)	$(0 + 2 \cdot 1, 1) = (2, 1)$
(0, -6)	$(0 + 2 \cdot 6, -6) = (12, -6)$
(-2, -6)	$(-2 + 2 \cdot 6, -6) = (10, -6)$

After this mapping the transformed points are still not linearly separable: plotting the six mapped points still does not allow a single straight line to separate all negatives from all positives.

2) Transformation $\text{phi}_2([x_1, x_2]) = [x_1, x_2 + (1/2)x_1^2]$

Original point	Mapped point under phi_2
(5, -3)	$(5, -3 + 0.5 \cdot 25) = (5, 9.5)$
(-6, -3)	$(-6, -3 + 0.5 \cdot 36) = (-6, 15)$
(2, 7)	$(2, 7 + 0.5 \cdot 4) = (2, 9)$
(0, 1)	$(0, 1 + 0) = (0, 1)$
(0, -6)	$(0, -6 + 0) = (0, -6)$
(-2, -6)	$(-2, -6 + 0.5 \cdot 4) = (-2, -4)$

Now the three negative examples all have transformed second coordinate values around 9 to 15, while the three positive examples have transformed second coordinate values 1, -6, and -4. So a simple horizontal separator such as $x_2' = 5$ separates the classes perfectly. Therefore the data **are linearly separable after phi_2**.

Question 2 conclusion

Original data: **not linearly separable**.
 After phi_1: **still not linearly separable**.
 After phi_2: **linearly separable**.

Question 3 - Agglomerative hierarchical clustering using similarity

The similarity between two objects is the number of matching attributes among Weight, Material, Size, Pattern, and Shape. The similarity between two clusters is the similarity of the **most similar cross-cluster pair**, which is the single-link analogue expressed in terms of similarity instead of distance.

Object	1	2	3	4	5
1	-	2	0	4	2
2		-	0	2	3
3			-	1	1
4				-	1
5					-

Iteration	Active clusters	Best similarity	Merge performed
1	{1}, {2}, {3}, {4}, {5}	4, from objects 1 and 4	Create cluster 6 = {1,4}

Iteration	Active clusters	Best similarity	Merge performed
2	{6}, {2}, {3}, {5}	3, from objects 2 and 5	Create cluster 7 = {2,5}
3	{6}, {7}, {3}	2, because $\text{sim}(6,7)=\max\{2,2,2,1\}=2$	Create cluster 8 = {1,4,2,5}
4	{8}, {3}	1, because $\text{sim}(8,3)=\max\{0,1,0,1\}=1$	Create cluster 9 = {1,2,3,4,5}

Question 3 conclusion

The pairwise similarities are: $s(1,2)=2$, $s(1,3)=0$, $s(1,4)=4$, $s(1,5)=2$, $s(2,3)=0$, $s(2,4)=2$, $s(2,5)=3$, $s(3,4)=1$, $s(3,5)=1$, $s(4,5)=1$. The merge sequence is **(1,4)**, then **(2,5)**, then **(6,7)**, then **(3,8)**.

Question 4 - MDP model for the commuter problem

An MDP is defined by $\langle S, A, T(s'|s,a), R(s,a) \rangle$. Here the states are the towns, the actions are the route choices or forced onward moves, the transition model captures the uncertainty at town E, and the only positive reward is for reaching town G.

State set

$S = \{A, B, C, D, E, F, G\}$. State G is terminal.

Available actions

State	Available action(s)
A	take_route_1, take_route_2
B	go_to_C
C	go_to_D
D	go_to_G
E	board_from_E
F	go_to_G
G	stop (or no-op, since G is terminal)

Non-zero transition probabilities

Transition	Probability
$T(B A, \text{take_route_1})$	1
$T(E A, \text{take_route_2})$	1
$T(C B, \text{go_to_C})$	1
$T(D C, \text{go_to_D})$	1
$T(G D, \text{go_to_G})$	1
$T(F E, \text{board_from_E})$	p
$T(A E, \text{board_from_E})$	1 - p
$T(G F, \text{go_to_G})$	1
$T(G G, \text{stop})$	1 (optional terminal self-loop)

Rewards

If rewards are written in the form $R(s,a)$, the only non-zero rewards are the actions that move directly into G:

$$\begin{aligned} R(D, \text{go_to_G}) &= 100 \\ R(F, \text{go_to_G}) &= 100 \end{aligned}$$

All other state-action rewards are 0. The discount factor γ can then be used later to penalise longer journeys.

Question 4 conclusion

This MDP captures exactly the situation described in the question: deterministic progress along route 1, a stochastic loop at state E on route 2, and reward only for successful arrival at G.

Question 5 - Effect of increasing p in the commuter MDP

When p increases, the train chosen at E is more likely to be the correct one to F. Therefore the transition model changes in the favorable direction: $T(F|E, \text{board_from_E})=p$ increases and $T(A|E, \text{board_from_E})=1-p$ decreases.

State-action values from A

Let γ be the discount factor and let the reward 100 be received on the last step into G.

$$Q(A, \text{take_route_1}) = 100 * \gamma^3$$

because route 1 takes exactly 4 legs: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$, so the reward is received after three discountings.

$$Q(A, \text{take_route_2}) = 100 * \gamma^2 * p / (1 - (1-p)*\gamma^2)$$

Explanation: a successful $A \rightarrow E \rightarrow F \rightarrow G$ trip gives reward after 3 legs, but each failed attempt at E sends the commuter back to A and adds a 2-leg loop $A \rightarrow E \rightarrow A$ before trying again. This produces the geometric series $100*\gamma^2*p[1 + ((1-p)\gamma^2) + ((1-p)\gamma^2)^2 + \dots]$.

Since increasing p both raises the success probability and reduces the loop-back probability, $Q(A, \text{take_route_2})$ increases with p , while $Q(A, \text{take_route_1})$ does not change.

If the two Q-values become equal

Equality of Q-values does **not** mean the routes are completely equivalent. It only means they have the same expected discounted return. Route 1 still has deterministic 4-hour travel time, whereas route 2 has random travel time because the commuter may loop back through A several times. So route 2 still has higher variability and more uncertainty, even if its expected discounted value matches route 1.

Question 5 conclusion

Increasing p makes route 2 more attractive by increasing the probability of progressing from E to F. However, equal Q-values imply equality of *expected discounted reward*, not equality of travel-time certainty, variance, or reliability.

Question 6 - Bias, variance, noise, and error as model complexity changes

Quantity	How it changes	Why
Bias ²	Decreases with complexity	More flexible models can fit the true signal better, so systematic error falls.
Variance	Increases with complexity	Flexible models react more strongly to fluctuations in the training sample.
Noise / irreducible error	Independent of complexity	Noise comes from the data-generating process, not the chosen model class.
Training error	Decreases with complexity	A more complex model can fit the training set at least as well as a simpler one.
Test error	Decreases and then increases	Initially the bias reduction helps, but after some point the rising variance causes overfitting.

Question 6 conclusion

The classical bias-variance picture is: **bias² down, variance up, noise unchanged, training error down, and test error down then up.**

Question 7 - Why training and testing must use different data

The correct choice is the one stating that using the same data for training and evaluation would very likely **overestimate** the true accuracy on unseen data.

A model has already adapted to the patterns, quirks, and noise in its training set. If we evaluate on that same set, the estimate is optimistically biased because the model is not being tested on genuinely new examples. Therefore option **(a)** is correct.

Question 8 - Reliability issue in the Python model-selection code

The main issue is **selection bias / optimistic evaluation caused by non-nested model selection**. The code first runs GridSearchCV on the full data set to choose the best hyperparameters, and then uses cross_val_score on that already-selected model using the same overall data set again.

Why this is a problem

The hyperparameters are chosen after seeing all data, including examples that later appear inside the evaluation folds. So the reported cross-validation accuracy is no longer a clean estimate of performance on unseen data. It is usually too optimistic because the search process has already adapted the model choice to the full sample.

How to fix it

Use **nested cross-validation**: an outer cross-validation loop for unbiased evaluation, and an inner GridSearchCV loop for tuning C and gamma only on the training part of each outer fold. This cleanly separates model selection from model assessment. Another acceptable alternative is to keep a final untouched test set that is used only once after tuning is complete.

```
Outer CV: estimate generalisation performance
Inner CV: tune hyperparameters on each outer-training split
```

Question 8 conclusion

The code leaks information from the full data set into the model-selection step. Nested cross-validation, or tuning on a validation set followed by a one-time test-set evaluation, is the correct remedy.